

Lectura de tarjeta NFC/RFID con ESP32 + PN532 (salida en Base 2, 10 y 16)

Fecha: 10/04/2026

Integrantes:

- Eduardo Cadengo López
- Itzel Citlalli Martell De La Cruz
- Damian Alexander Diaz Piña

Índice

1) Datos generales.....	1
2) Descripción de la tecnología RFID / NFC	1
3) Usos actuales de la tecnología.....	2
4) Funcionamiento y descripción del módulo sensor (PN532)	2
5) Diagrama de conexión (ESP32 ↔ PN532 por I2C)	2
6) Código utilizado.....	3
7) Evidencias (fotos + video).....	6
8) Link del repositorio en GitHub	6

1) Datos generales

- **Práctica:** Lectura de UID en bases Binaria, Decimal y Hexadecimal
- **Plataforma:** ESP32
- **Módulo sensor:** PN532 (NFC) por I2C
- **Entorno:** Arduino IDE
- **Objetivo:** Mostrar en el monitor serial el UID de una tarjeta en **Base 16, Base 10 y Base 2.**

2) Descripción de la tecnología RFID / NFC

RFID (Radio Frequency Identification) es una tecnología de identificación inalámbrica que usa ondas de radio para leer/escribir información en una etiqueta (tag) mediante un lector.

Existen distintos rangos de frecuencia, y en alta frecuencia (HF) se trabaja comúnmente a **13.56 MHz**.

NFC (Near Field Communication) es una forma “especial” de RFID enfocada a comunicaciones **de muy corto alcance** (tipo “acercar y leer”), muy utilizada en pagos sin contacto y smartphones. En la práctica, NFC opera dentro de estándares como **ISO/IEC 14443** (proximity cards), donde el alcance típico es de pocos centímetros.

En tarjetas NFC/RFID HF, el lector genera un campo electromagnético (13.56 MHz) y la tarjeta pasiva aprovecha ese campo para energizarse y responder, enviando su **UID (identificador único)** y otros datos según el tipo de tarjeta/estándar.

3) Usos actuales de la tecnología

Algunos usos comunes y reales de RFID/NFC incluyen:

- **Control de acceso** (credenciales para puertas, oficinas, laboratorios).
- **Pagos sin contacto** (tarjetas bancarias contactless / billeteras NFC).
- **Transporte público** (tarjetas de metro/bus con validación rápida).
- **Identificación y trazabilidad** (inventarios, bibliotecas, logística, etc.).

4) Funcionamiento y descripción del módulo sensor (PN532)

El **PN532** es un controlador NFC muy usado para lectura/escritura de tarjetas a **13.56 MHz**. Es capaz de trabajar como **lector/escritor** para tecnologías como **ISO/IEC 14443A/MIFARE**, y también soporta otros modos y protocolos (según configuración y firmware). [\[nxp.com\]](https://www.nxp.com),

Además, el PN532 puede comunicarse con un microcontrolador mediante varias interfaces: **I2C, SPI o UART** (dependiendo del módulo/breakout y configuración). [\[nxp.com\]](https://www.nxp.com),

En esta práctica se usó **I2C**, que es un bus de dos líneas:

- **SDA**: datos
- **SCL**: reloj

Y requiere resistencias pull-up (muchos módulos ya las incluyen).

5) Diagrama de conexión (ESP32 ↔ PN532 por I2C)

5.1 Conexiones típicas I2C

En Arduino-ESP32 (ESP32 genérico), los pines I2C más comunes por defecto son:

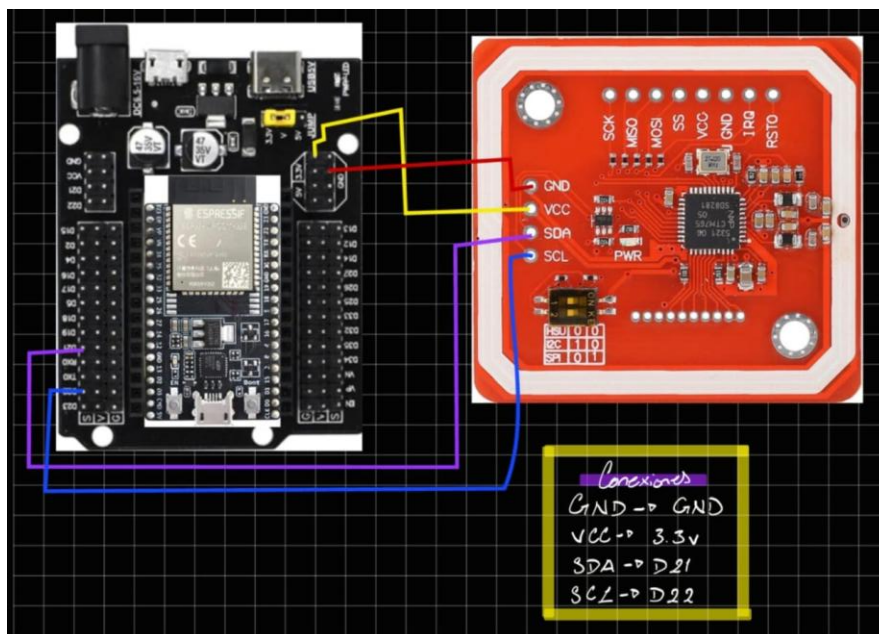
- **GPIO21 = SDA**
- **GPIO22 = SCL**

5.2 Tabla de conexión

- ESP32 3V3 → PN532 VCC (si tu módulo acepta 3.3V)
- ESP32 GND → PN532 GND
- ESP32 GPIO21 (SDA) → PN532 SDA
- ESP32 GPIO22 (SCL) → PN532 SCL

La librería de Adafruit contempla I2C y funciones para leer tarjetas tipo ISO14443A/MIFARE mediante llamadas como `readPassiveTargetID(...)`. [github.com], [docs.arduino.cc]

5.3 Diagrama (ASCII)



6) Código utilizado

```
#include <Wire.h>
#include <Adafruit_PN532.h>

// ----- Pines I2C (ESP32) -----
// En ESP32 genérico: SDA=21 y SCL=22 suelen ser los más usados. (Se pueden
cambiar)
#define SDA_PIN 21
#define SCL_PIN 22

// OJO: Tu proyecto lo trae así. Si a ti te compila y funciona, lo dejamos
igual.
Adafruit_PN532 nfc(SDA_PIN, SCL_PIN);
```

```

// ----- Funciones simples para imprimir el UID -----
void imprimirHex(const uint8_t *uid, uint8_t len) {
    Serial.print("HEX (Base 16): ");
    for (uint8_t i = 0; i < len; i++) {
        if (uid[i] < 0x10) Serial.print("0"); // cero a la izquierda
        Serial.print(uid[i], HEX);
        if (i < len - 1) Serial.print(":");
    }
    Serial.println();
}

void imprimirDec(const uint8_t *uid, uint8_t len) {
    Serial.print("DEC (Base 10): ");
    for (uint8_t i = 0; i < len; i++) {
        Serial.print(uid[i], DEC);
        if (i < len - 1) Serial.print("-");
    }
    Serial.println();
}

void imprimirBin(const uint8_t *uid, uint8_t len) {
    Serial.print("BIN (Base 2): ");
    for (uint8_t i = 0; i < len; i++) {
        // siempre 8 bits por byte
        for (int b = 7; b >= 0; b--) {
            Serial.print((uid[i] >> b) & 1);
        }
        if (i < len - 1) Serial.print(" ");
    }
    Serial.println();
}

void setup() {
    Serial.begin(115200);
    delay(50);

    Serial.println("Iniciando PN532-");

    // Iniciar I2C con los pines elegidos (en ESP32 se recomienda hacerlo
    explícito)
    Wire.begin(SDA_PIN, SCL_PIN);

    nfc.begin();

```

```

// Verifica que el módulo esté respondiendo
uint32_t version = nfc.getFirmwareVersion();
if (!version) {
    Serial.println("ERROR: PN532 no encontrado. Revisa conexiones");
    while (1) { delay(10); }
}

// Configura el módulo para lectura (SAM)
nfc.SAMConfig();

Serial.println("PN532 listo. Acerca una tarjeta...\n");
}

void loop() {
    uint8_t uid[7];    // buffer típico (UID hasta 7 bytes en varios casos)
    uint8_t uidLen = 0;

    // Lee una tarjeta tipo ISO14443A (MIFARE/NFC-A).
    // El último parámetro es timeout (ms). Si no detecta, regresa false.
    bool encontrada = nfc.readPassiveTargetID(PN532_MIFARE_ISO14443A, uid,
    &uidLen, 1000);

    if (encontrada) {

        Serial.println("  TARJETA DETECTADA");

        // Aquí está lo importante: mismo UID, tres representaciones
        imprimirHex(uid, uidLen);    // Base 16
        imprimirDec(uid, uidLen);    // Base 10
        imprimirBin(uid, uidLen);    // Base 2

        Serial.println("\n");

        delay(2000); // pausa para que no se imprima mil veces la misma tarjeta
    }
}

```

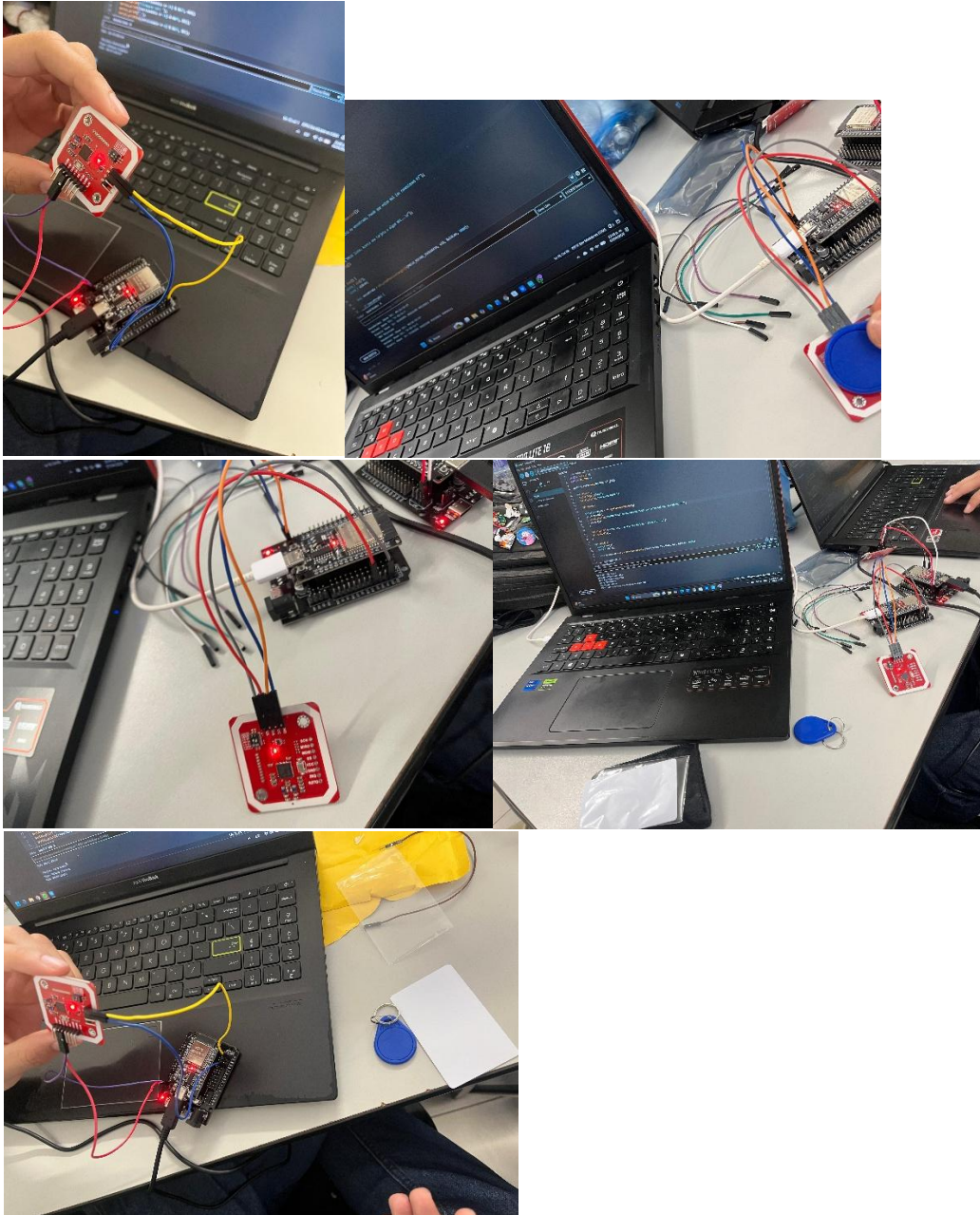
¿Qué hace el código?

1. Inicializa Serial y el PN532.
2. Espera una tarjeta ISO14443A.
3. Lee el UID y lo imprime en **Hex (Base16)**, **Dec (Base10)** y **Bin (Base2)**.

4. El objetivo se cumple cuando el monitor serial muestra el UID en las tres bases.

7) Evidencias (fotos + video)

Link: <https://youtu.be/XhOumT1LdeU>



8) Link del repositorio en GitHub

Link:

https://github.com/AresGodKiller/Tecnologias_Inalambricas/tree/main/NFC_Basics

9) Conclusiones

Eduardo: Se logró cumplir el objetivo de la práctica, ya que el sistema detecta la tarjeta con el PN532 y muestra el UID correctamente en hexadecimal, decimal y binario en el monitor serial. Esto demuestra que la comunicación entre la ESP32 y el módulo funciona de forma estable y que el formato del dato puede adaptarse a distintas necesidades de identificación.

Itzel: Esta práctica ayudó a entender cómo una misma identificación (UID) puede representarse en diferentes bases numéricas, lo cual es útil cuando se requiere depuración, registro de usuarios o compatibilidad con sistemas que manejan distintos formatos (por ejemplo, validaciones por HEX o por decimal). Además, se comprobó que la lectura es rápida y repetible al acercar la tarjeta al sensor.

Damian: La implementación con PN532 permitió observar el proceso completo de lectura NFC y reforzó la importancia de una buena conexión I2C (SDA/SCL) y configuración del módulo para evitar fallas de comunicación. Como resultado, se obtuvo un método confiable para identificar tarjetas que puede servir como base para proyectos posteriores como control de acceso o autenticación.